

```
import numpy as np
import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
```

```
df = pd.read_csv('https://raw.githubusercontent.com/gscdit/Breast-Cancer-Detection/refs/heads/master/data.csv')
df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_m
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1

5 rows × 33 columns

```
df.shape
```

```
(569, 33)
```

```
df.drop(columns=['id', 'Unnamed: 32'], inplace= True)
```

```
df.head()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	point
0	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	C
1	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	C
2	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	C
3	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	C
4	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	C

5 rows × 31 columns

✖ train test split

```
X_train, X_test, y_train, y_test = train_test_split(df.iloc[:, 1:], df.iloc[:, 0], test_size=0.2)
```

✖ scaling

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
X_train
```

```
array([[ -6.50565842e-01, -1.48908266e-01, -5.76498693e-01, ...,
         6.15575005e-01,  2.86214223e+00,  2.99362329e+00],
       [  5.97634634e-01,  9.02319666e-01,  4.91580943e-01, ...,
        -2.20860954e-01,  3.81162965e-02,  4.72656715e-06],
       [ -1.58044534e+00, -8.13451034e-01, -1.52901703e+00, ...,
        -1.40800994e-01,  3.84791784e-03,  9.23682295e-01],
       ...,
       [ -3.54939414e-01,  6.91610984e-01, -2.60429911e-01, ...,
         1.27282915e+00,  7.53079287e-01,  2.13876454e+00],
       [  6.96176777e-01,  9.13897066e-01,  7.63236241e-01, ...,
         1.68609104e+00,  2.96027440e+00,  6.24750555e-01],
       [ -2.95216903e-01, -2.13741707e-01, -3.74697615e-01, ...,
        -9.77541944e-01, -1.45878879e+00, -1.22744340e+00]])
```

y_train

	diagnosis
31	M
10	M
114	B
487	M
28	M
...	...
534	B
318	B
229	M
370	M
37	B

455 rows × 1 columns

dtype: object

Label Encoding

```
encoder = LabelEncoder()
y_train = encoder.fit_transform(y_train)
y_test = encoder.transform(y_test)
```

y_train

```
array([1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0,
       0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0,
       0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0,
       1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0,
       0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1,
       0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0,
       1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1,
       0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0])
```

Numpy arrays to PyTorch tensors

```
X_train_tensor = torch.from_numpy(X_train)
X_test_tensor = torch.from_numpy(X_test)
y_train_tensor = torch.from_numpy(y_train)
y_test_tensor = torch.from_numpy(y_test)
```

X_train_tensor.shape

torch.Size([455, 30])

y_train_tensor.shape

torch.Size([455])

Defining the model

```
class MySimpleNN():
    def __init__(self, X):
```

```

self.weights = torch.rand(X.shape[1], 1, dtype=torch.float64, requires_grad=True)
self.bias = torch.zeros(1, dtype=torch.float64, requires_grad=True)

def forward(self, X):
    z = torch.matmul(X, self.weights) + self.bias
    y_pred = torch.sigmoid(z)
    return y_pred

def loss_function(self, y_pred, y):
    # Clamp predictions to avoid log(0)
    epsilon = 1e-7
    y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)

    # Calculate loss
    loss = -(y_train_tensor * torch.log(y_pred) + (1 - y_train_tensor) * torch.log(1 - y_pred)).mean()
    return loss

```

Important Parameters

```

learning_rate = 0.1
epochs = 25

```

Training Pipeline

```

# create model
model = MySimpleNN(X_train_tensor)

# define loop
for epoch in range(epochs):

    # forward pass
    y_pred = model.forward(X_train_tensor)

    # loss calculate
    loss = model.loss_function(y_pred, y_train_tensor)

    # backward pass
    loss.backward()

    # parameters update
    with torch.no_grad():
        model.weights -= learning_rate * model.weights.grad
        model.bias -= learning_rate * model.bias.grad

    # zero gradients
    model.weights.grad.zero_()
    model.bias.grad.zero_()

    # print loss in each epoch
    print(f'Epoch: {epoch + 1}, Loss: {loss.item()}')

```

```

Epoch: 1, Loss: 3.8734547280441722
Epoch: 2, Loss: 3.747472466032905
Epoch: 3, Loss: 3.6134186400812918
Epoch: 4, Loss: 3.470439804709892
Epoch: 5, Loss: 3.322739972220773
Epoch: 6, Loss: 3.1694648115531123
Epoch: 7, Loss: 3.0100440752635733
Epoch: 8, Loss: 2.844157902489334
Epoch: 9, Loss: 2.674609873272524
Epoch: 10, Loss: 2.4944661028022916
Epoch: 11, Loss: 2.310878252311848
Epoch: 12, Loss: 2.1262855749988945
Epoch: 13, Loss: 1.9355652547438864
Epoch: 14, Loss: 1.7455326544884913
Epoch: 15, Loss: 1.5644006244127746
Epoch: 16, Loss: 1.3939717578405724
Epoch: 17, Loss: 1.240793872530291
Epoch: 18, Loss: 1.1079115183532575
Epoch: 19, Loss: 0.9976743132283318
Epoch: 20, Loss: 0.9111688924041779
Epoch: 21, Loss: 0.8476079601805322
Epoch: 22, Loss: 0.8040883970545488
Epoch: 23, Loss: 0.776156415200584
Epoch: 24, Loss: 0.759000203855423
Epoch: 25, Loss: 0.7485464460500285

```

```
model.bias
```

```
tensor([-0.0940], dtype=torch.float64, requires_grad=True)
```

▼ Evaluation

```
# model evaluation
with torch.no_grad():
    y_pred = model.forward(X_test_tensor)
    y_pred = (y_pred > 0.9).float()
    accuracy = (y_pred == y_test_tensor).float().mean()
    print(f'Accuracy: {accuracy.item()}')
```

Accuracy: 0.6378886103630066

Start coding or [generate](#) with AI.